



Weaver Efficient Multi-LLM Serving with Attention Offloading

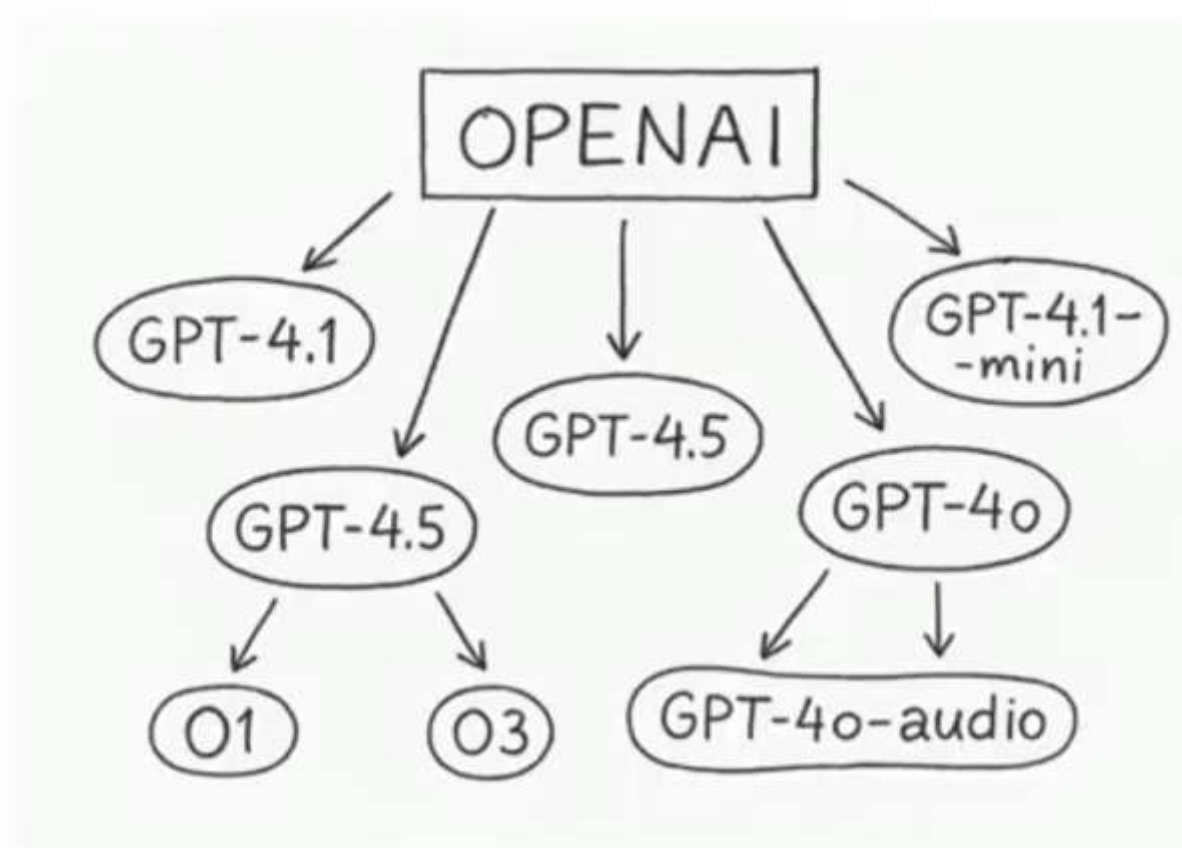
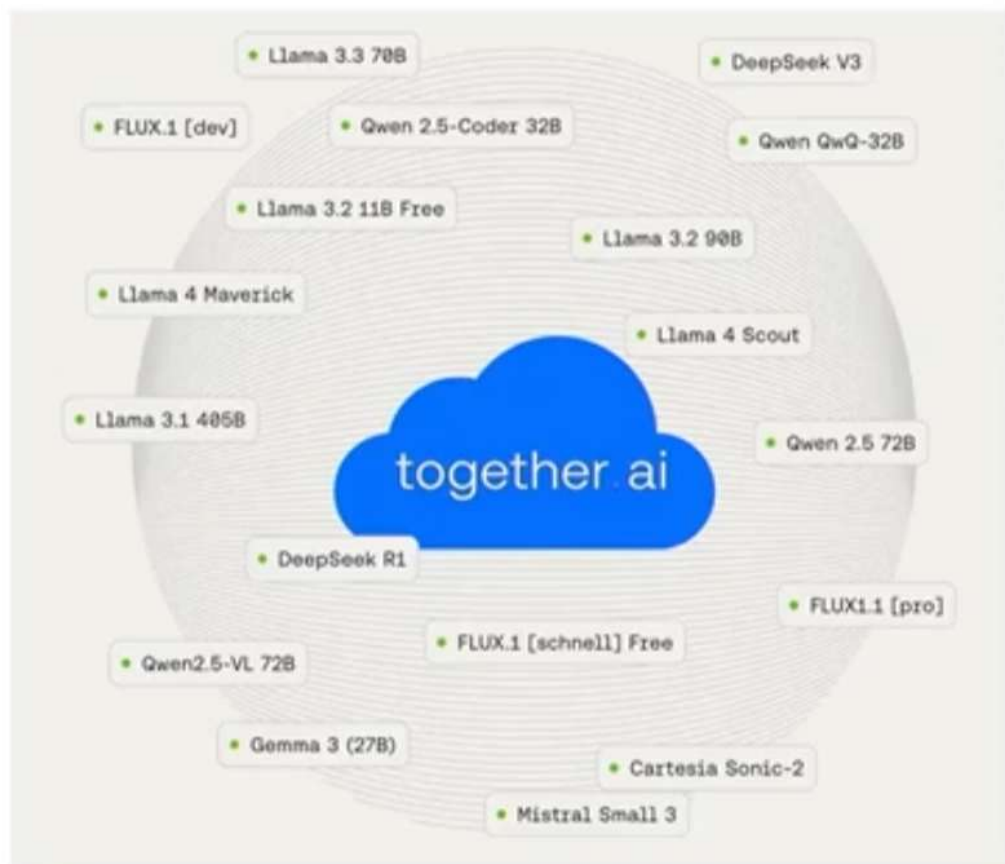
ATC'25

Shiwei Gao, Qing Wang, Shaoxun Zeng, Youyou
Lu, Jiwu Shu



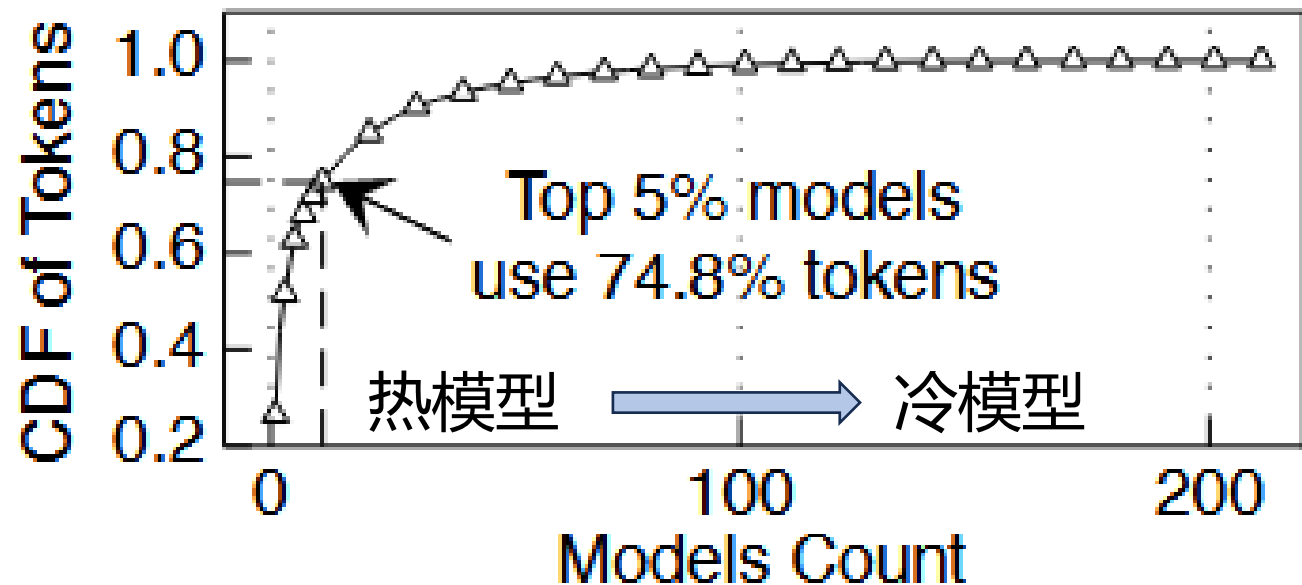
汇报人：洪雪怀
2026.5.18

背景-如今的大模型平台

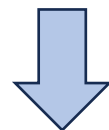


云平台需同时维护数十乃至上百个不同的大模型

问题1-热模型的困境

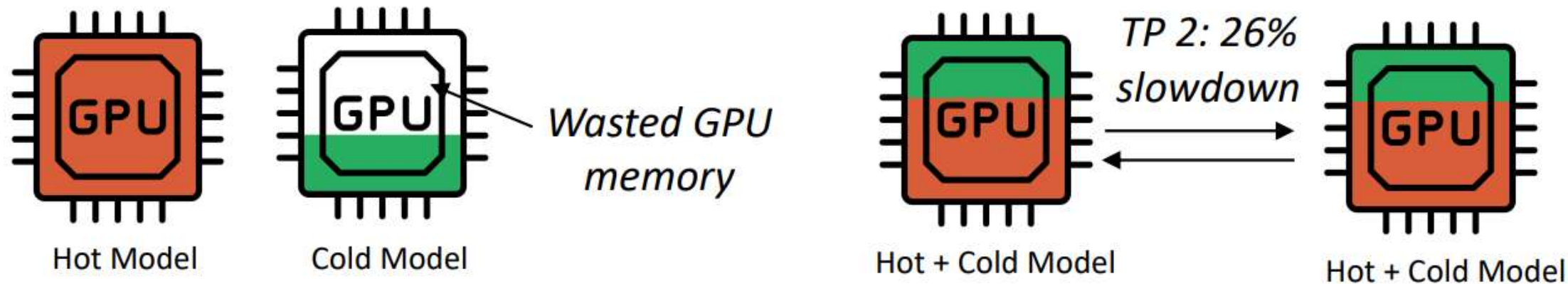


头部5%的模型占据了75%的负载



热模型的显存墙与冷模型的显存荒废

相关工作-专属GPU与张量并行



专属GPU

产生显存浪费

张量并行

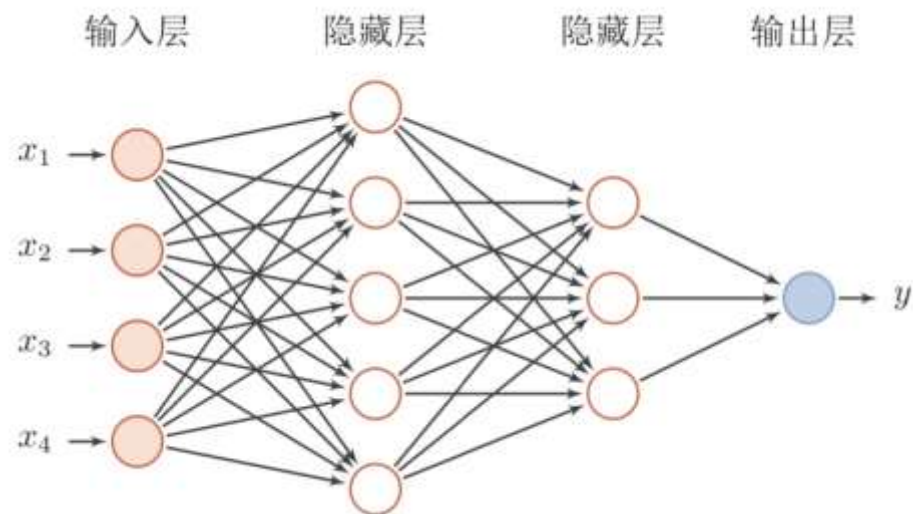
产生通信开销

问题2-张量并行的通信开销

	专用GPU	张量并行	
Batch Size	2 × Seperate (Token/s)	TP 2 (Token/s)	Slowdown
16	2024	1487	26.6%
32	3343	2645	20.9%
64	5392	4443	17.6%
128	8011	6606	17.5%

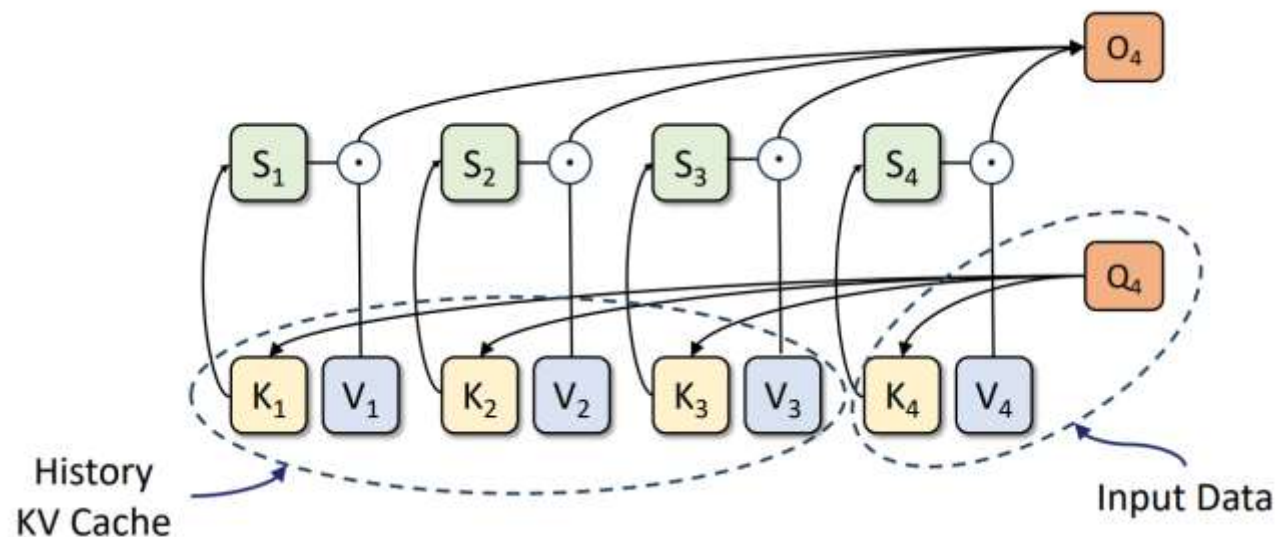
张量并行架构相比于专用GPU架构吞吐量全面下降！

背景-算子的类别



参数密集型算子-FNN

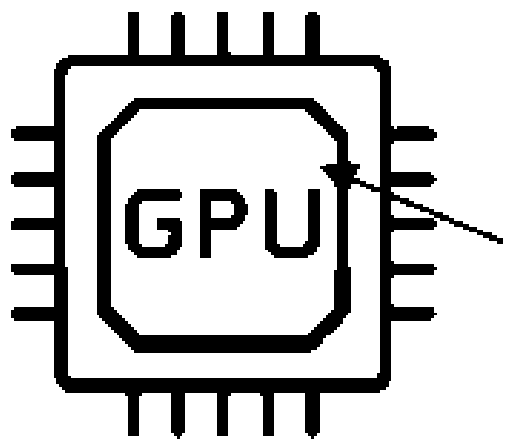
需要频繁传输中间结果



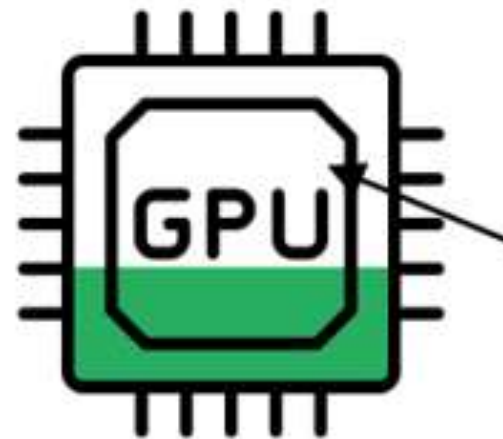
参数稀疏型算子-注意力

只需要传输新的QKV向量

相关工作-注意力卸载



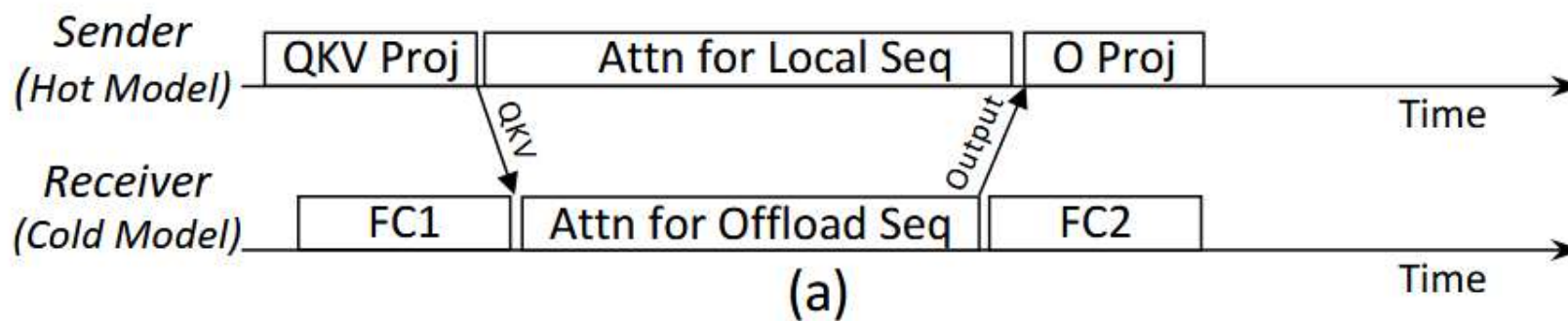
卸载至专属GPU
LoongServe



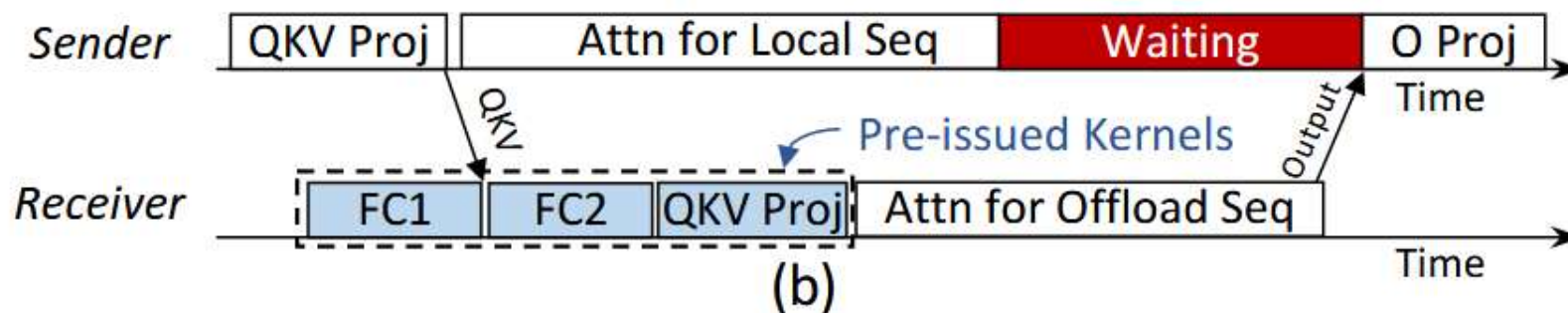
卸载至冷模型的GPU
Weaver

问题3-冷模型的算子阻塞

理想

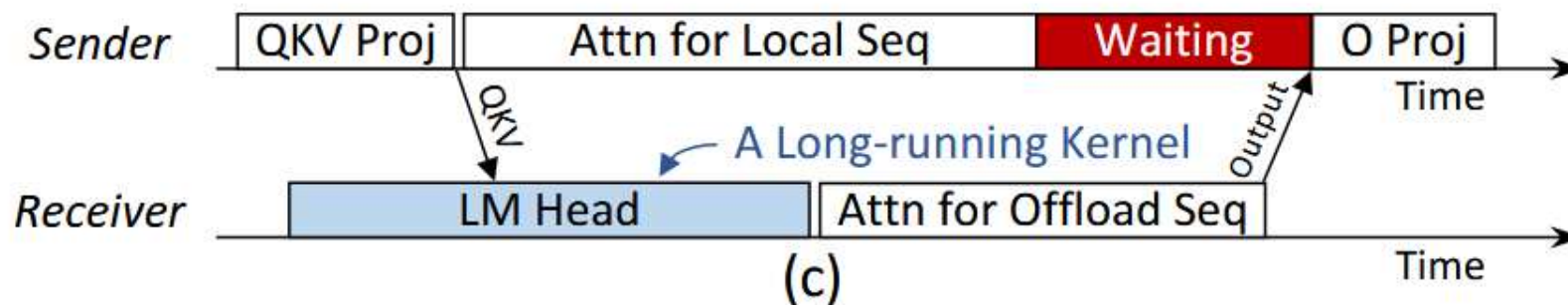


预发射
核函数



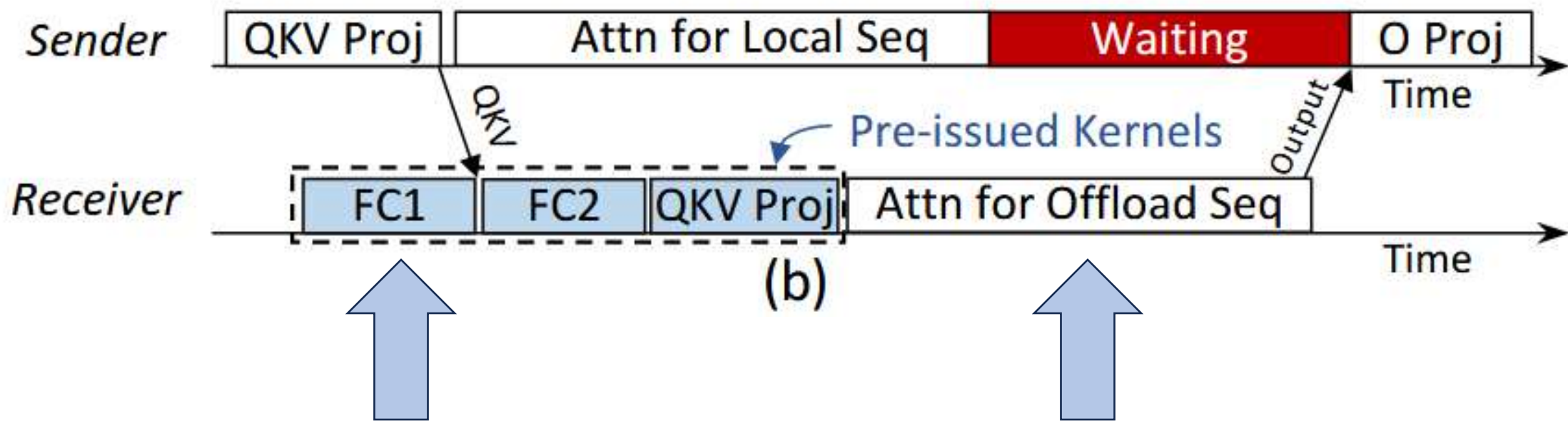
多任务
阻塞

长发射
核函数



长任务
阻塞

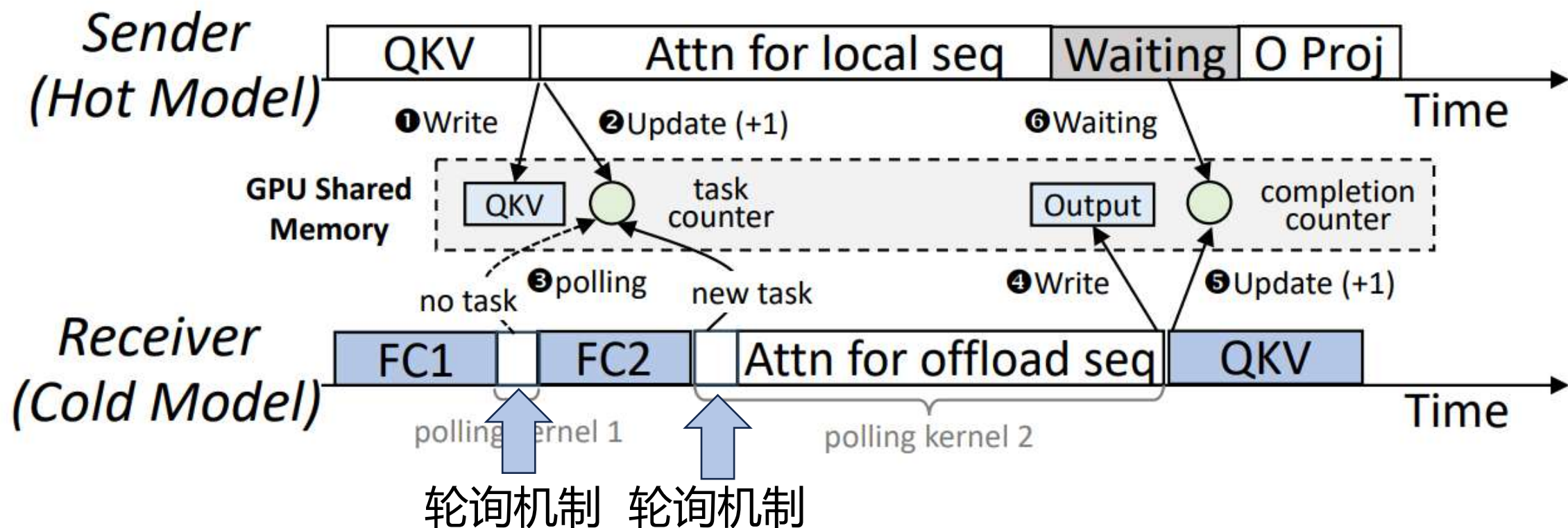
设计-如何解决预发射核函数问题？



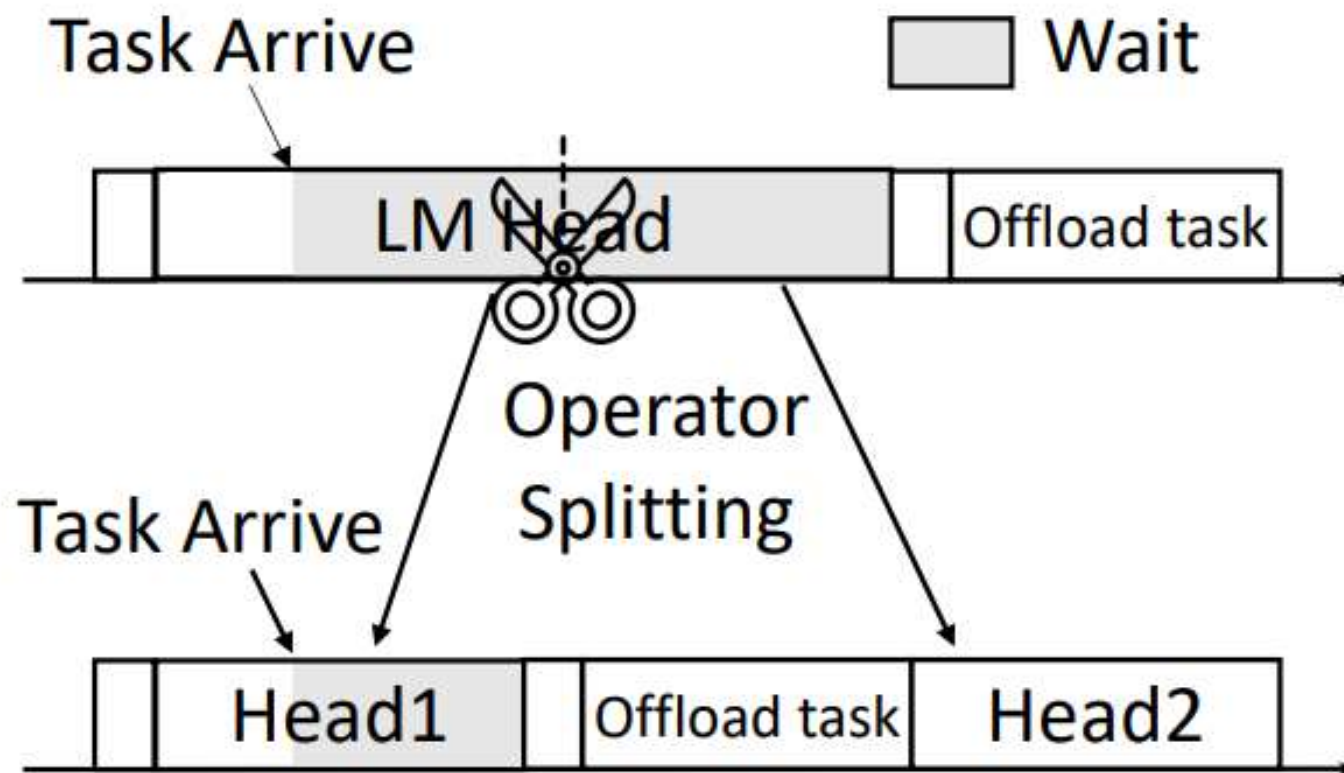
CPU发布的任务

让注意力任务插队

设计-GPU驱动的具体流程

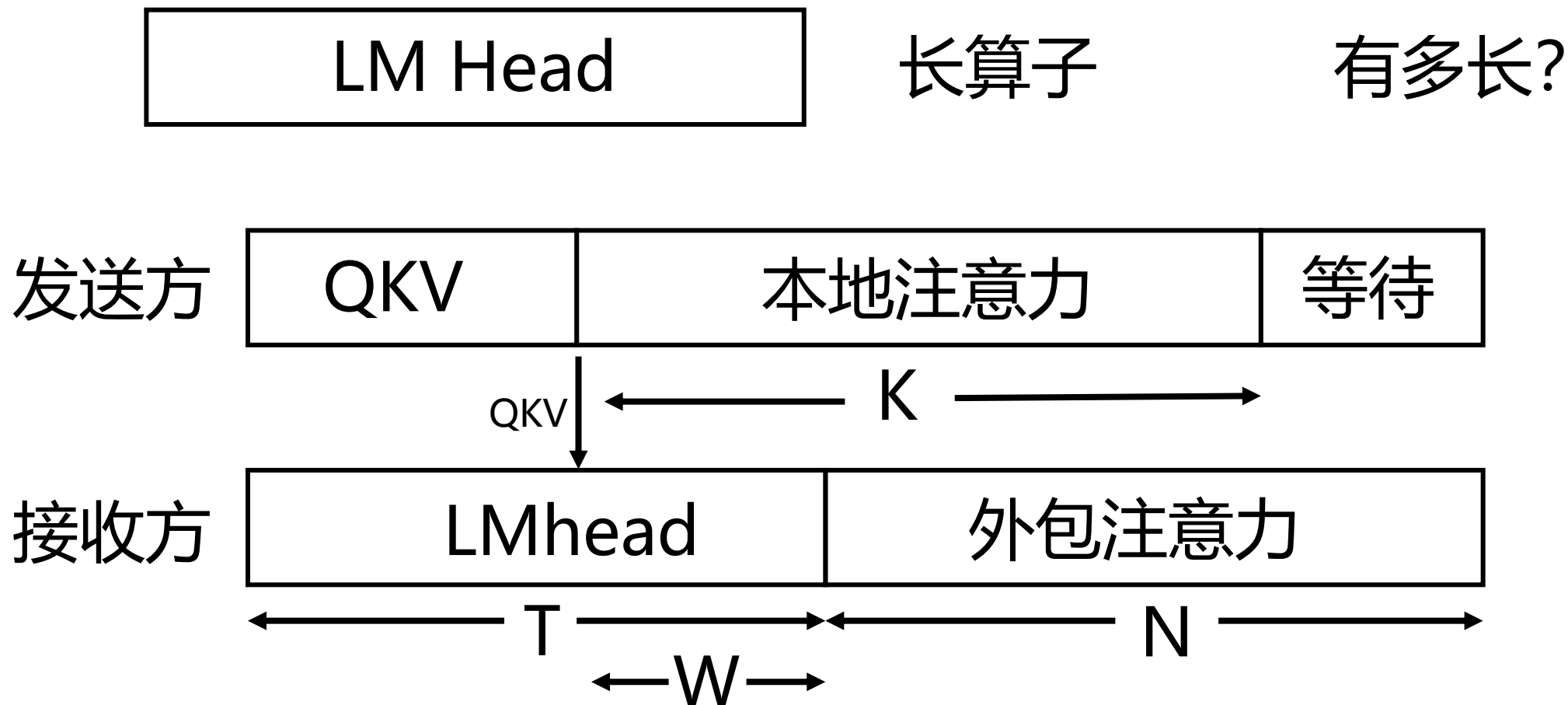


设计-如何解决长耗时核函数问题？

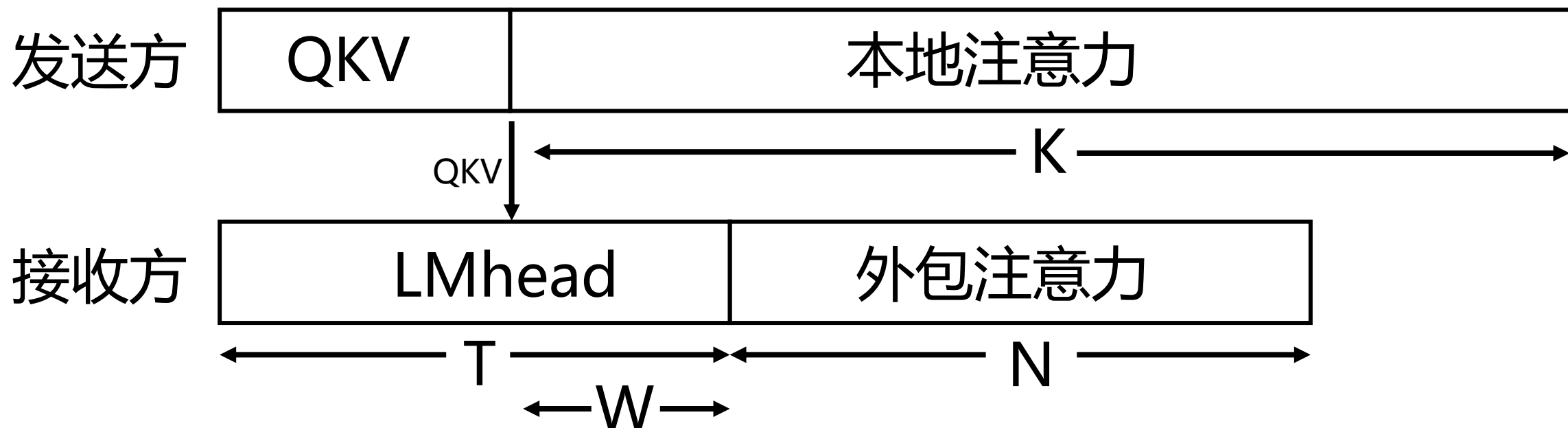


算子拆分

设计-对什么算子进行拆分？



设计-对什么算子进行拆分？



$W + N < K$ 的情况发送方不会察觉到等待
运行过程中不能拆分算子

$$W < K - N \quad \rightarrow \quad W < T < K - N$$

设计-算子拆分的具体流程

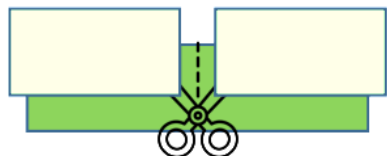


根据算子耗时进行排序



第一次判断：等待时间 < 门限？

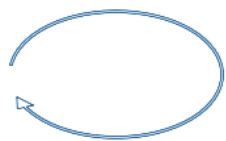
第二次判断：耗时 > $K - N$ ？



将耗时最长的算子拆分



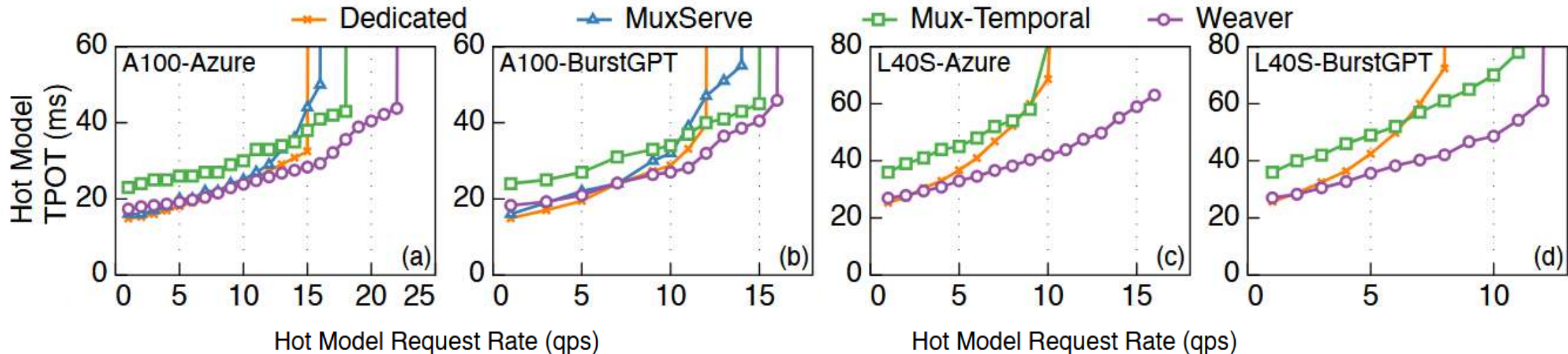
将生成的算子放入队列



Algorithm 1 Priority-Based Operator Splitting.

```
1: Input: Priority queue  $Q$ , which contains all operators  
   reverse order of  $T_i$ ; Wait time threshold  $WT_{\text{threshold}}$ .  
2: Output: A queue of operators after splitting.  
3: while Estimated wait time >  $WT_{\text{threshold}}$  do  
4:    $OP_{\max} \leftarrow Q.\text{front}();$   
5:   if  $T_{\max} > K - N$  then  
6:      $OP_1, OP_2 \leftarrow \text{split}(OP_{\max});$   
7:      $Q.\text{pop}();$   
8:      $Q.\text{push}(OP_1, OP_2);$   
9:   else  
10:    break  
11: return  $Q$ 
```

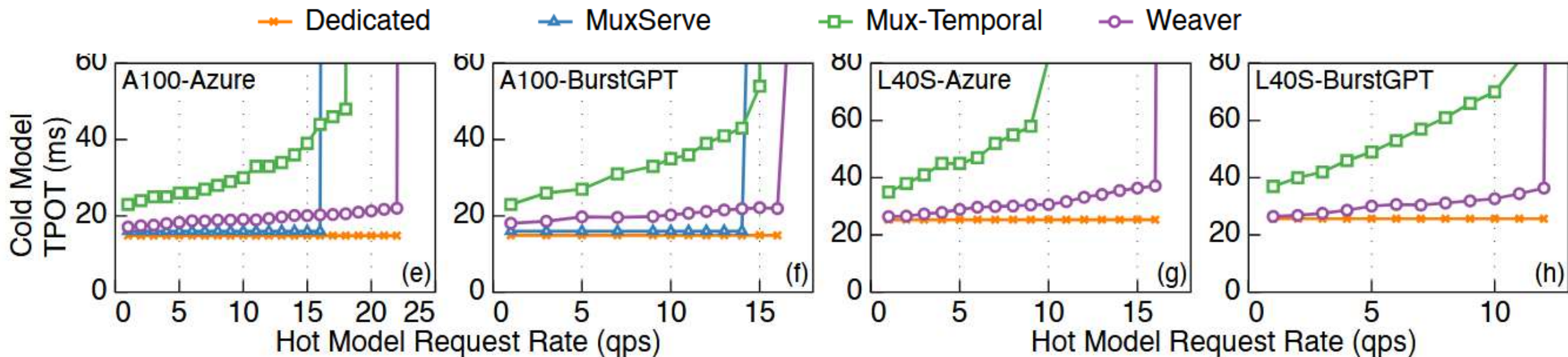
结果-热模型在不同系统的效率对比



吞吐量大幅提升

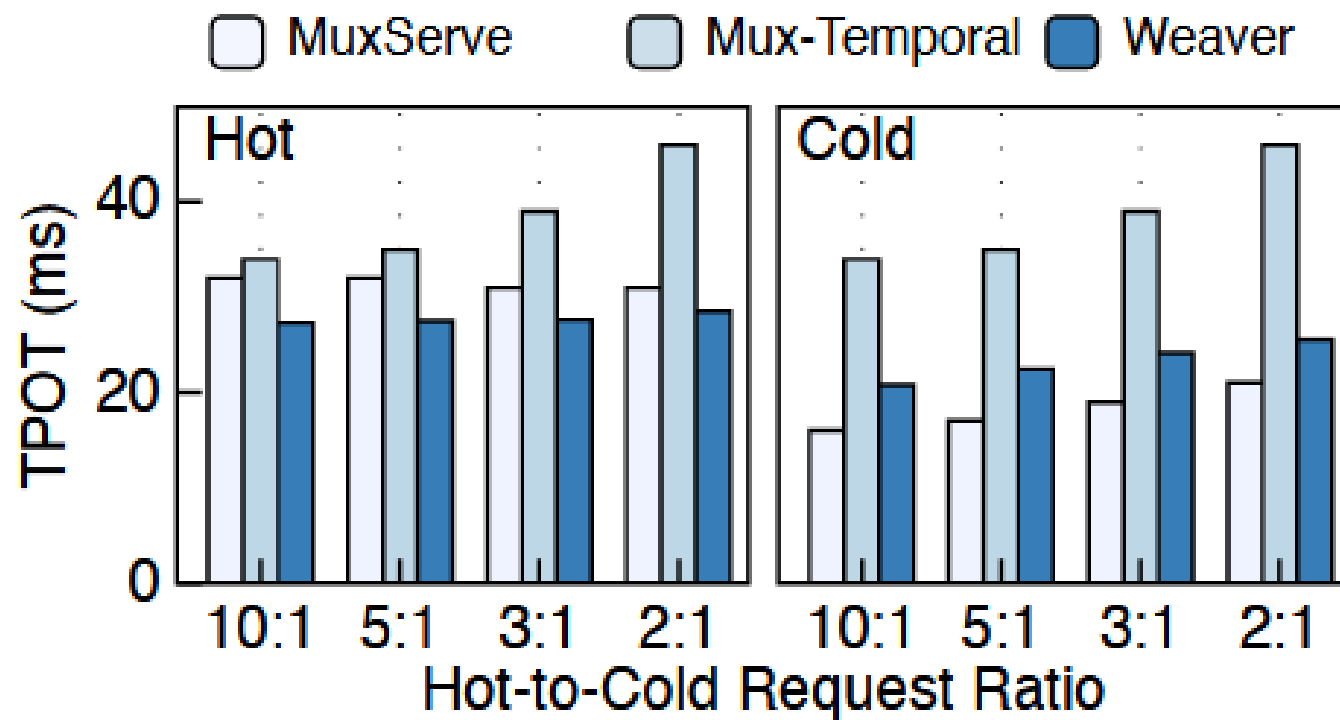
适应弱网环境

结果-冷模型在不同系统的效率对比



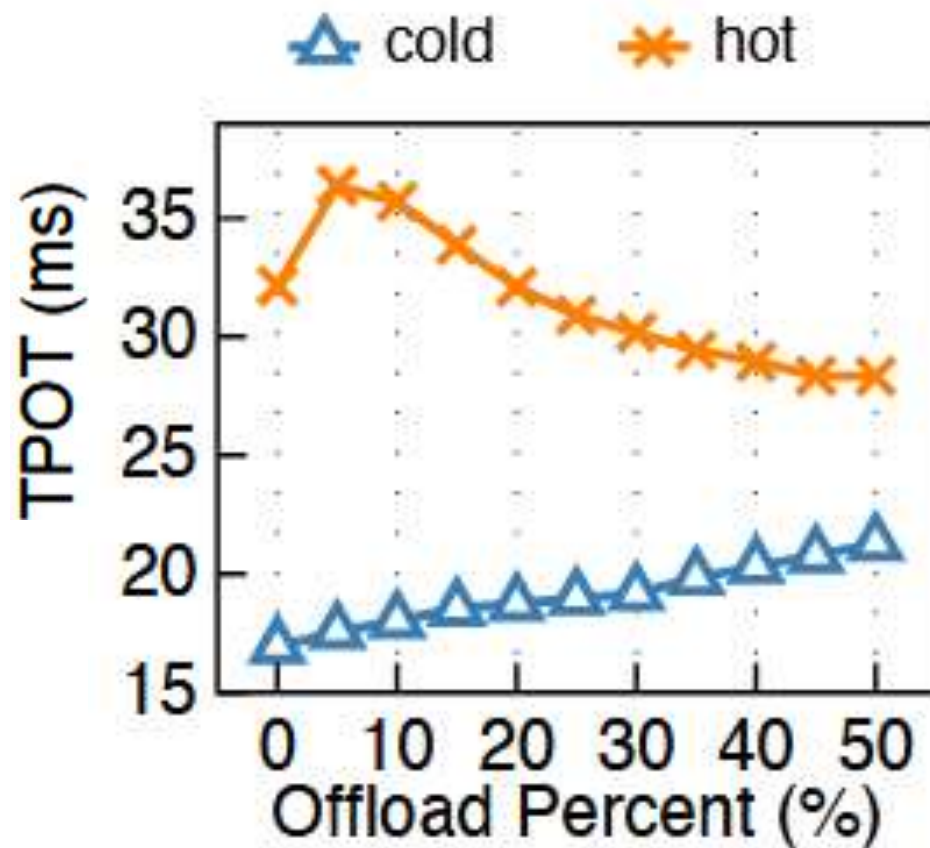
Weaver--极低的延迟干扰

结果-敏感度实验



Weaver--适应各种请求比例

结果-什么是合适的卸载比例？



45%最合适

更小?—卸载注意力无意义

更大?—发送方会频繁等待

思考

改进方向

- 1.引入实时的请求率指标，当冷模型负载变高时动态降低卸载比例，当冷模型空闲时再调高，实现动态卸载
- 2.探索如何对含有参数的 FFN 部分也进行高效的工作负载卸载。

泛化方向

- 1.在多智能体协作网络当中，智能体也呈现冷热交替的现象，引入注意力卸载可以充分利用冷智能体的显存。
- 2.MoE 架构存在天然的“冷热专家”分布，可以让高负载的“热专家”把 Attention 计算卸载给集群中闲置的“冷专家”。



Q & A

汇报人：洪雪怀
2026.5.18